Introduction to Programming - Fall 2026

Functions

Write a piece of code once. Give it inputs. Use its result.

Write our own functions, use functions from libraries, and build a small Pokemon battle game.

WEEK 7 - FUNCTIONS

Part 1: Writing functions

From a named block of code to a result you can use.

01	Quick recap
02	What is a function?
03	Defining and calling
04	Parameters and arguments
05	Return values
06 / 07	Execution and scope

Practice: draw a face, check a shopping budget, and trace a function.

01 - Quick recap

We can store several values and work through them with a loop.

```
scores = [12, 18, 24]          # A list
person = ("Mina", 20)          # A tuple

total = 0
for score in scores:
    total += score

print(total)                   # 54
print(person[0])               # Mina
```

Today, we will put code like this inside **functions**.

02 - What is a function?

A named piece of code that performs a task. You already call functions:

```
name = input("Your name: ")
print(name)
print(len(name))
```

Function	What it does
`input()`	Reads text and gives it back to us
`print()`	Displays something
`len()`	Gives us the length of something

Some functions take inputs. Some give us a useful result.

Why write our own?

Imagine using the same code in several places.

- Give that task a clear name.
- Call it whenever you need it.
- Change its code in one place instead of many.

```
show_menu()
```

A name such as `show_menu` tells us what the code is for. Next, we will learn how to write that function.

03 - Defining and calling

Defining writes the instructions. Calling runs them.

```
def show_menu():
    print("1. Play")
    print("2. Quit")
```

Part	Meaning
`def`	Start a function definition
`show_menu`	The name we choose
`()`	Where parameters will go
`:`	The indented block starts next
Indented lines	The **body**: instructions indented by four spaces

A definition is not a call

```
def show_menu():
    print("1. Play")
    print("2. Quit")

print("Welcome!")
show_menu()
print("Choose an option.")
```

```
Welcome!
1. Play
2. Quit
Choose an option.
```

Python runs the body only when it reaches `show_menu()`.

Call it again

```
def show_menu():
    print("1. Play")
    print("2. Quit")

show_menu()
show_menu()
```

Each call starts the body again.

Write the definition before the first call runs. Use parentheses to call a function: `show_menu()`, not just `show_menu`.

04 - Parameters and arguments

Run the same instructions with different inputs.

```
def show_score(score):
    print(f"Score: {score}")

show_score(12)
show_score(30)
```

```
Score: 12
Score: 30
```

Parameter: the name in the definition, such as `score`.

Argument: the value supplied by a call, such as `12`.

More than one input

```
def show_score(name, score):
    print(f"{name}: {score}")

show_score("Mina", 12)
show_score("Sam", 30)
```

Arguments are matched to parameters **in order**.

You can also use their names:

```
show_score(score=12, name="Mina")
```

These are **keyword arguments**. The names must match the definition.

A default value

```
def show_score(name, score=0):
    print(f"{name}: {score}")

show_score("Mina")       # Mina: 0
show_score("Sam", 30)    # Sam: 30
```

The default is used only when that argument is omitted.

Put parameters without defaults before parameters with defaults.

Exercise: Draw a face · 10 min

Write `draw_face(eyes, mouth)`.

- Print the eyes twice, with a space between them.
- Print the mouth on the next line, with one space before it.
- Call it twice with different characters. Choose your own!

Examples:

`draw_face("o", "-")`

```
o o
 -
```

`draw_face("^", "v")`

```
^ ^
 v
```

Folder: exercises/07-functions/draw-a-face/

Solution: Draw a face

```
def draw_face(eyes, mouth):
    print(f"{eyes} {eyes}")
    print(f" {mouth}")

draw_face("o", "-")
draw_face("^", "v")
```

```
o o
 -
^ ^
 v
```

The function stays the same; its arguments change the face.

05 - Return values

Give a result back to the code that called the function.

```
def double(number):
    return number * 2

points = double(8)
print(points)               # 16
print(points + 5)           # 21
```

`double(8)` gives back `16`.

The code making the call is the **caller**. It can save the returned value, calculate with it, or use it in a condition.

`print` and `return` are different

Display a value

```
def show_double(number):
    print(number * 2)

result = show_double(8)
print(result)
```

```
16
None
```

Give back a value

```
def double(number):
    return number * 2

result = double(8)
print(result + 5)
```

```
21
```

Without an explicit `return`, a function returns `None`: no useful value.

`return` ends the call

```
def ticket_price(age):
    if age < 12:
        return 5
    return 10

print(ticket_price(8))       # 5
print(ticket_price(16))      # 10
```

Once Python reaches a `return`, it goes back to the caller.

For age `8`, the second `return` is never reached.

Exercise: Shopping budget · 10 min

You are building the code for a wallet. The wallet starts with **20 euros**. You will buy different items with it and check whether you can afford them.

1. Write `total_price(unit_price, quantity)`. It **returns** a number: the cost of **quantity** items at **unit_price** each.
2. A notebook costs 3 euros. Print the total for 4 and for 8 notebooks.
3. **(Advanced)** Make a basket: a list of `(price, quantity)` tuples. Write `can_afford(shopping_basket, wallet_amount)`. It **returns** `True` if the wallet can pay for the whole basket, otherwise `False`.

Expected input and output

```
basket = [(3, 4), (2, 3)]
total_price(3, 4)       # returns 12
total_price(3, 8)       # returns 24
can_afford(basket, 20)  # returns True
can_afford(basket, 10)  # returns False
```

Help: loop over a list of tuples

```
for unit_price, quantity in basket:
    print(unit_price, quantity)
```

Starter: exercises/07-functions/shopping-budget/

Solution: Shopping budget

```
def total_price(unit_price, quantity):
    return unit_price * quantity

print(total_price(3, 4))
print(total_price(3, 8))

def can_afford(shopping_basket, wallet_amount):
    total = 0
    for unit_price, quantity in shopping_basket:
        total += total_price(unit_price, quantity)
    return total <= wallet_amount
```

`can_afford` reuses the value that `total_price` returns. If `total_price` only printed it, we could not add it up.

06 - How a function runs

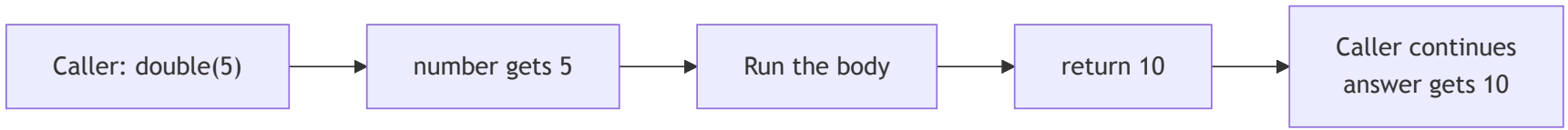
Follow a call from start to finish.

```
def double(number):
    result = number * 2
    return result

answer = double(2 + 3)
print(answer)
```

1. Python defines `double`; it does not run the body yet.
2. It calculates the argument: `2 + 3` gives `5`.
3. It enters the function: the parameter `number` has value `5`.
4. It calculates `result`, then returns `10`.
5. The caller saves `10` in `answer` and prints it.

The path of a call



The caller waits for the function's result before continuing.

If the body reaches its end without a `return`, the result is `None`.

One function can call another

```
def double(number):
    return number * 2

def double_plus_one(number):
    result = double(number)
    return result + 1

answer = double_plus_one(3)
print(answer)                # 7
```

`double_plus_one(3)` calls `double(3)` inside its body.

`double(3)` returns `6`. Then `double_plus_one` adds `1` and returns `7`. Both functions can also be called on their own.

07 - Variable scope

Scope means where a variable name can be used.

```
def double(number):
    result = number * 2
    return result

answer = double(4)
print(answer)               # 8
```

`number` and `result` are **local**: their names belong to this function call.

`answer` belongs to the code outside the function. Returning `8` does not make the name `result` available outside.

A local name is not available outside

```
def double(number):
    result = number * 2
    return result

answer = double(4)
print(result)                # NameError
```

The caller received the **value**, not the function's local variable name.

Use `answer` here, not `result`.

Same name, different variables

```
score = 10                  # A global name: outside functions

def show_score():
    score = 3               # A local name: inside this call
    print(score)

show_score()                # 3
print(score)                # 10
```

Assignment inside the function normally creates a local variable. It does not reassign the global `score`.

Each call has its own local names

```
def first_point():
    points = 0
    points += 1
    return points

print(first_point())         # 1
print(first_point())         # 1
```

The second call starts again at `points = 0`.

Local variables are not shared between separate calls.

Reading a global name

```
school = "ReDI"

def show_school():
    print(school)

show_school()                # ReDI
```

A function can read a name defined outside it.

Usually, prefer an explicit input: `show_school(school)`. Parameters and return values make a function's inputs and outputs easier to see.

Trace execution and scope · 5 min

```
value = 10

def add_two(value):
    result = value + 2
    print(result)
    return result

answer = add_two(1)
print(answer)
print(value)
```

What three lines are printed? Could we use `result` outside the function?

Task and answers are in the lesson's exercise and solution READMEs.

Solution: Execution and scope

```
3
3
10
```

- Inside the call, the parameter `value` is `1`, so `result` is `3`.
- The caller saves the returned `3` in `answer`.
- The global `value` stays `10`.
- `result` is a local name. Using it outside would cause a `NameError`.

Break

15 minutes

Part 2: Reusing code

Document our functions, use libraries, and build a game.

08	Docstrings
09	Libraries and imports
10	Installing with pip
11	Mini project: Pokemon battle
12	Homework and questions

08 - Docstrings

A **docstring** is a short description inside a function. It is the first statement in the body.

```
def double(number):
    """Return twice the supplied number."""
    return number * 2
```

Use triple quotes. Describe what the function does.

For longer functions, also explain their inputs and returned result.

Read a function's help

```
def double(number):
    """Return twice the supplied number."""
    return number * 2

help(double)
```

Python shows the function's name, parameters, and docstring.

Notice `help(double)`, **not** `help(double(4))`. We want help about the function, not about its returned number.

Comments and docstrings

```
def circle_area(radius):
    """Return the area of a circle with this radius."""
    # The calculation goes here.
    pass
```

- A **docstring** tells callers what the function is for.
- A **comment** can explain a particular line of code.
- `pass` is a placeholder that does nothing. Replace it with working code.

You will add docstrings to both functions in the battle project.

09 - Libraries and imports

We do not have to write everything ourselves.

A **library** is reusable code supplied for other programs to use.

A **module** is a unit of Python code we can import, often a `.py` file. A library can contain several modules.

Python includes a **standard library**. It is installed with Python.

Module	Useful for
`math`	Mathematical functions and constants
`random`	Random choices and numbers

Import a module

```
import math

print(math.sqrt(25))         # 5.0
print(round(math.pi, 2))     # 3.14
```

`import math` makes the module available in this program.

A module has **members**: the names defined inside it. Some are functions, like `math.sqrt`. Some are values, like `math.pi`.

The dot means "use this member of that module": `math.sqrt` is the square-root function in `math`.

Import a function by name

```
from math import sqrt

print(sqrt(25))             # 5.0
```

Now we can call `sqrt()` without writing `math.`. Import only the names you need. Avoid `from math import *`.

Shorter name: `as` gives an import an **alias**, another name to use.

```
import random as rand

print(rand.randint(1, 6))
```

Random choices and numbers

```python
import random

names = ["pikachu", "bulbasaur", "charmander"]
opponent = random.choice(names)
damage = random.randint(5, 15)

print(opponent)
print(damage)
```

`choice()` picks one item from a non-empty sequence.

`randint(5, 15)` gives an integer from **5 through 15**, including both ends.

The output can change each time we run the program.

Exercise: Lunch picker · 10 min

You and your friends can never decide what to eat for lunch. You will write a program that picks a lunch for you at random.

1. Import `random`. Write `pick_lunch(options)`. It takes a list of foods and **returns** one of them, chosen at random.
2. Make a list called `foods` with at least three foods. Print the lunch that `pick_lunch` chooses from it.
3. **(Advanced)** Write `plan_week(options, days)`. It **returns** a list with one random lunch for each day, so the list has `days` items. Use `pick_lunch` inside it.

Expected input and output (random)

```
foods = ["pizza", "sushi", "falafel"]
pick_lunch(foods)
# returns "sushi" (or another food)
plan_week(foods, 3)
# returns ["pizza", "pizza", "falafel"]
```

Your program prints one food, e.g. `sushi`

Help

```
import random
print(random.choice(["heads", "tails"]))
# heads or tails
```

Starter: exercises/07-functions/lunch-picker/

Solution: Lunch picker

```
import random

def pick_lunch(options):
    return random.choice(options)

foods = ["pizza", "sushi", "falafel"]
print(pick_lunch(foods))

def plan_week(options, days):
    lunches = []
    for day in range(days):
        lunches.append(pick_lunch(options))
    return lunches
```

Python already supplies `random`; we do not need to install it.

10 - Installing with pip

Install a library that does not come with Python, then import it.

pip installs Python packages. A **package** is an installable bundle of code and other files.

We will use `pokebase` to fetch real Pokemon data.

- Install it once on your computer.
- Import it in each program that uses it.
- Installation and fetching the data need internet access.

Installing a package does **not** run it in your program. Importing a missing package causes `ModuleNotFoundError`.

Install `pokebase`

Open your exercise folder in VS Code and choose **Terminal → New Terminal**.

Windows (PowerShell)

```
py -m pip install pokebase==1.4.1
```

Then run your code:

```
py exercise.py
```

macOS

```
python3 -m pip install pokebase==1.4.1
```

Then run your code:

```
python3 exercise.py
```

`-m pip` installs the package for the same Python that runs your code.

Use `pokebase` in our code

```
import pokebase

pokemon = pokebase.pokemon("pikachu")

print(pokemon.name)          # pikachu
print(pokemon.height)        # 4
print(pokemon.weight)        # 60
```

The library handles downloading and reading the data for us.

`pokebase.pokemon()` is a library function. It **returns** an object with the Pokemon's data. We read each piece of data with a dot, such as `pokemon.height`.

Exercise: Pokemon stats · 10 min

You are building a Pokedex app. Players want to know how strong each Pokemon is. A Pokemon's strength is the total of its six base stats.

1. Import `pokebase`. Write `total_stats(name)`. It fetches the Pokemon and **returns** the total of its six base stats.
2. Print the totals for Pikachu and Bulbasaur.
3. **(Advanced)** Write `strongest(names)`. It takes a list of names and **returns** the name with the highest total. Use `total_stats` inside it.

Expected input and output

```
total_stats("pikachu")
# returns 320
strongest(["bulbasaur", "pikachu"])
# returns "pikachu"
```

Your program prints: `320` then `318`

Help: each stat has a `base_stat` number

```
for stat in pokemon.stats:
    print(stat.base_stat)
# 35, 55, 40, 50, 50, 90
```

Starter: exercises/07-functions/pokemon-stats/

Solution: Pokemon stats

```
import pokebase

def total_stats(name):
    pokemon = pokebase.pokemon(name)
    total = 0
    for stat in pokemon.stats:
        total += stat.base_stat
    return total

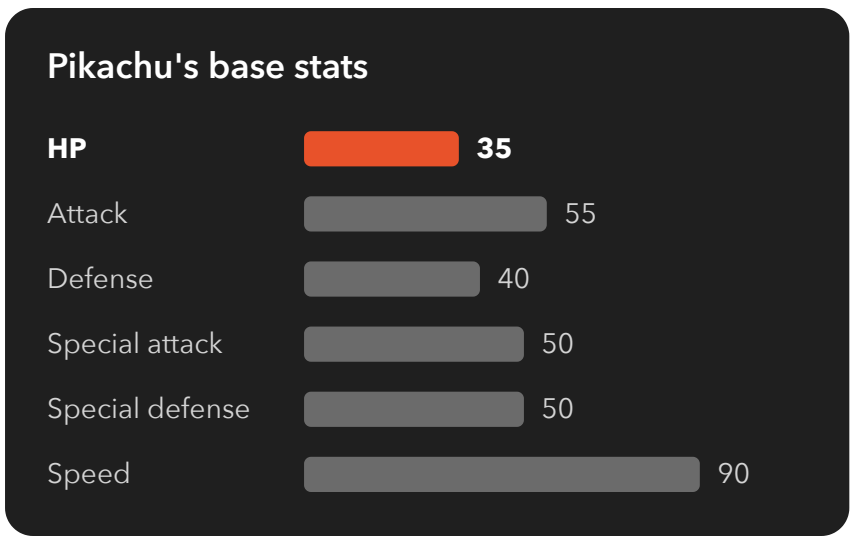
print(total_stats("pikachu"))    # 320
print(total_stats("bulbasaur"))  # 318

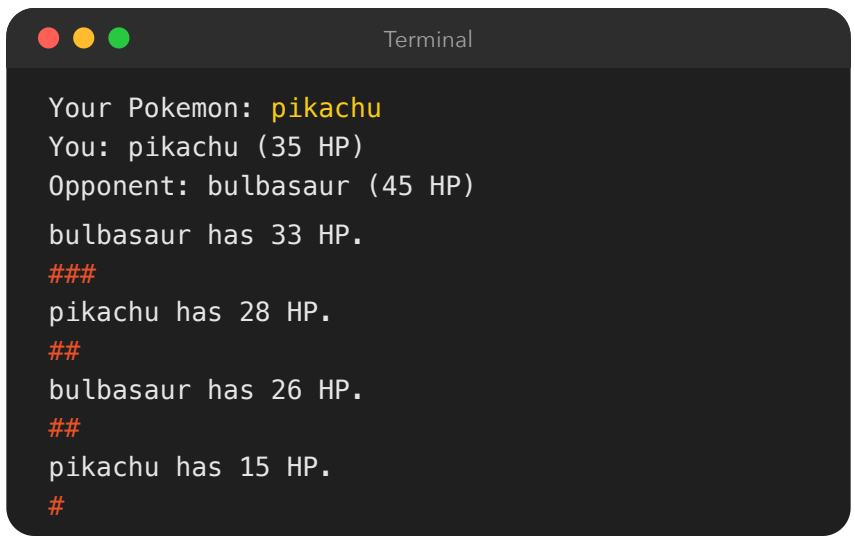
def strongest(names):
    best = names[0]
    for name in names:
        if total_stats(name) > total_stats(best):
            best = name
    return best
```

11 - Mini project: Pokemon battle

Every Pokemon has six **base stats**. The first one is **HP** (health points): how much damage it can take before it loses.

In our game, you pick a Pokemon and the computer picks an opponent. The program then simulates the battle: they take turns hitting each other for a random amount of damage, so luck decides who wins. The first Pokemon to reach 0 HP loses.





Exercise: Pokemon battle · 20 min

Copy the code from the starter: mini-project-pokemon-battle/exercise.py. It already runs the game; your functions make it work.

1. Write `get_hp(name)`. It fetches the Pokemon and **returns** its starting HP.
2. Write `attack(hp, damage)`. It **returns** the HP left after the hit, never below 0.
3. In the game loop, use `attack` to update `opponent_hp` and `player_hp`.
4. **(Advanced)** Write `health_bar(hp)`. It **returns** one `#` for every full 10 HP. Print it after each hit.

Expected input and output

```
get_hp("pikachu")   # returns 35
attack(35, 10)      # returns 25
attack(5, 10)       # returns 0
health_bar(35)      # returns "###"
```

Help: the first stat is HP

```
pokemon = pokebase.pokemon(name)
print(pokemon.stats[0].base_stat)
```

Solution: The functions

```
def get_hp(name):
    pokemon = pokebase.pokemon(name)
    return pokemon.stats[0].base_stat

def attack(hp, damage):
    remaining = hp - damage
    if remaining < 0:
        return 0
    return remaining

def health_bar(hp):
    bar = ""
    for i in range(hp // 10):
        bar += "#"
    return bar
```

Solution: The battle loop

```
    while player_hp > 0 and opponent_hp > 0:
        damage = random.randint(5, 15)
        opponent_hp = attack(opponent_hp, damage)
        print(f"{opponent_name} has {opponent_hp} HP.")
        print(health_bar(opponent_hp))

        if opponent_hp > 0:
            damage = random.randint(5, 15)
            player_hp = attack(player_hp, damage)
            print(f"{player_name} has {player_hp} HP.")
            print(health_bar(player_hp))
```

This loop is inside `main()`. The full solved program is in the matching `solutions/` folder.

12 - Homework

Four short tasks. Each has an `exercise.py` and its own `check.py`.

Task	Practise
1. Ticket prices	Parameters, conditions and returning a price
2. High scores	A list parameter and a loop
3. Average score	Returning a calculation for the caller to use
4. Roll a die	Default parameters, a library and a docstring

Folder: exercises/07-functions/

Homework 1: Ticket prices

Write `ticket_price(age)`.

- Return `5` if the age is below `12`.
- Otherwise return `10`.
- Print the returned prices for ages `8` and `16`.
- Add those two returned prices and print the total.

```
5
10
15
```

Folder: `homework-01-ticket-prices/`

Solution: Ticket prices

```
def ticket_price(age):
    if age < 12:
        return 5
    return 10

child = ticket_price(8)
adult = ticket_price(16)
print(child)
print(adult)
print(child + adult)
```

Homework 2: High scores

Write `count_high_scores(scores, minimum)`.

Use a loop to count scores **greater than or equal to** `minimum`. Return the count.

Use `[12, 45, 7, 30]` and a minimum of `20`. Print the result.

```
2
```

Folder: `homework-02-high-scores/`

Solution: High scores

```
def count_high_scores(scores, minimum):
    count = 0
    for score in scores:
        if score >= minimum:
            count += 1
    return count

print(count_high_scores([12, 45, 7, 30], 20))
```

Homework 3: Average score

Write `average(scores)`. Assume the list is non-empty.

Use a loop to find the total, then return `total / len(scores)`.

Call it with `[12, 18, 24]`. Print the result. If it is at least `15`, also print `Pass`.

```
18.0
Pass
```

Folder: `homework-03-average-score/`

Solution: Average score

```
def average(scores):
    total = 0
    for score in scores:
        total += score
    return total / len(scores)

result = average([12, 18, 24])
print(result)
if result >= 15:
    print("Pass")
```

Homework 4: Roll a die

Import `random`. Write `roll_die(sides=6)` with a docstring.

Return a random integer from `1` through `sides`, including both ends.

Print one roll using the default and one using `sides=20`.

One possible output:

```
4
17
```

Folder: `homework-04-roll-a-die/`

Solution: Roll a die

```
import random

def roll_die(sides=6):
    """Return a random integer from 1 through sides."""
    return random.randint(1, sides)

print(roll_die())
print(roll_die(sides=20))
```

The output changes. The checker tests the range and both endpoints.

The main ideas

- **Define** a function with `def`; **call** it with parentheses.
- **Parameters** name the inputs; **arguments** supply their values.
- **Return** gives a result to the caller; **print** displays something.
- **Scope** determines where a variable name is available.
- **Docstrings** describe what a function does.
- **Import** reusable code; **pip** installs additional packages.

Use small functions with clear inputs and useful results.

Questions?